Week 7 Report — Cost Optimization & Continuous Learning

Author: Arnav Mahale **Course:** AIPI 561 — Operationalizing AI **Date:** 2026-06-19

Overview

The final week makes the Week 5–6 TechCorp agent cheaper to run and able to improve from user feedback. Three systems live in `cost_optimization_starter.py`:

System	Class	What it does
Cost analysis	`CostAnalyzer`	Break each query's cost into components; flag expensive outliers
Optimization	`OptimizationStrategy`	Caching, retrieval trimming, model selection, response compression
Continuous learning	`FeedbackLoop`	Collect and validate user corrections; measure their impact

All three are pure Python with no LLM calls, so the test suite runs deterministically and offline.

Design

1. CostAnalyzer

- `record_query(query)` — stores a per-query cost breakdown across four components: `retrieval_cost`, `llm_cost`, `tool_cost`, `error_cost`. Missing components default to 0, `total_cost` is computed if absent, and a UTC timestamp is attached.
- `get_cost_breakdown()` — sums each component across all queries and reports the daily total and query count, so you can see *where* spend goes (LLM vs. retrieval vs. retries).
- `identify_cost_spikes()` — flags queries whose cost exceeds **mean + 2 · stdev**, the standard outlier rule. (This needs a realistic sample: with only a handful of queries a single extreme outlier inflates the standard deviation enough to hide itself, so the rule is applied over a full batch of queries.)

2. OptimizationStrategy

- `apply_caching(query, response)` — returns `(is_hit, response)`; a repeat query is served from cache and skips the LLM call entirely.
- `optimize_retrieval_count(n)` — trims retrieval to the top-3 documents, cutting the tokens sent to the model (e.g. 15 → 3).
- `select_model_by_complexity(query)` — routes simple lookups (`What is…`, `Who…`, `define…`) to the cheap `gemini-2.5-flash-lite`, sends analytical queries (`analyze`, `compare`, `design`, …) to `gemini-2.5-pro`, and defaults everything else to `gemini-2.5-flash`.
- `enable_response_compression(response)` — keeps only the first N sentences of a long answer.
- `get_optimization_impact()` — reports estimated % savings per applied strategy, capped to stay realistic (strategies have diminishing returns when stacked).

3. FeedbackLoop

- `submit_correction(query, original, corrected, role)` — validates and stores a correction. A correction is accepted only if the submitter has **manager-level authority or above** (engineer/hr/finance are below the bar) **and** the corrected answer is more detailed than the original. Every submission is stored with a `valid` flag so quality can be tracked over time.
- `validate_correction(index)` — re-checks a stored correction against the same rules.
- `get_feedback_metrics()` — reports total corrections, validation rate, average correction length, and the most common subjects of corrected queries (`top_error_patterns`).

Test Results — `python3 cost_optimization_starter.py`

```
Testing CostAnalyzer...
  breakdown: {'retrieval_total': 0.027, 'llm_total': 0.109, 'tool_total': 0.0095, 'error_tot
  get_cost_breakdown: PASSED
  spikes detected: ['Analyze every department budget for the year']
  identify_cost_spikes: PASSED

Testing OptimizationStrategy...
  first call (miss): (False, 'Pre-approval needed.')
  second call (hit): (True, 'Pre-approval needed.')
  apply_caching: PASSED
  optimize_retrieval_count(15) -> 3: PASSED
  simple  query -> gemini-2.5-flash-lite
  complex query -> gemini-2.5-pro
  select_model_by_complexity: PASSED
  compressed: 'Travel must be pre-approved. Domestic limit is $5000.'
```

```
  enable_response_compression: PASSED
  optimization impact: {'total_savings_pct': 85.0, 'strategies_applied': ['caching', 'retrie

Testing FeedbackLoop...
  engineer correction: {'accepted': False, 'reason': "role 'engineer' (level 1) lacks author
  manager correction: {'accepted': True, 'reason': 'accepted'}
  weak manager correction: {'accepted': False, 'reason': 'correction must be more detailed t
  submit_correction: PASSED
  validate_correction: PASSED
  feedback metrics: {'total_corrections': 3, 'validation_rate': 33.3, 'avg_correction_length
  get_feedback_metrics: PASSED

All tests passed!
```

Discussion

- **Cost breakdown** shows the LLM is the dominant cost (~$0.109 of $0.176), which is where optimization should focus.
- **Spike detection** correctly isolates the one runaway query (heavy retrieval + large LLM cost + retries) out of nine.
- **Optimization** stacks four strategies for an estimated ~85% reduction; caching alone removes the LLM call on repeat questions.
- **Feedback** enforces who may correct the agent and rewards detailed corrections; the 33% validation rate reflects that only one of three sample submissions met both bars.

Note on the model

Model selection returns Gemini tier names (`flash-lite` / `flash` / `pro`). The running agent from Weeks 5–6 executes on `gemini-2.5-flash` because the free tier sets `gemini-2.5-pro`'s quota to zero; `select_model_by_complexity` demonstrates the routing logic you would use with billing enabled.

Files

- `cost_optimization_starter.py` — `CostAnalyzer`, `OptimizationStrategy`, `FeedbackLoop` + tests
- `app_starter.py`, `access_control_starter.py` — the Week 5–6 agent this builds on
- `REPORT.md` / `REPORT.pdf` — this report